

Licitaciones Públicas de Chile

Pipeline de datos e inteligencia de mercado sobre las compras del Estado · Mercado Público / ChileCompra

Python · pandas · Streamlit · Altair · pydantic

API REST + Datos Abiertos masivos

dbt · DuckDB · GitHub Actions (CI/CD)

Daniel González Simpson · Junio 2026

1. Resumen ejecutivo

Este proyecto es un **pipeline de datos de punta a punta** y un **dashboard interactivo** sobre las licitaciones públicas de Chile. Combina dos fuentes que ninguna herramienta gratuita cruza: la **API en vivo de Mercado Público** (lo que está abierto para postular hoy) y los **datos abiertos masivos de ChileCompra** (cómo terminaron ~43.000 licitaciones de los últimos seis meses, a nivel de oferta por línea de producto).

El usuario objetivo es un **proveedor que quiere venderle al Estado**. El dashboard responde tres preguntas de manera progresiva: *¿qué oportunidades hay y cuándo cierran?* (exploración), *¿qué exige esta licitación específica?* (detalle en vivo con cronograma, ítems y montos), y la pregunta diferenciadora que el buscador oficial no responde: *¿cuánta competencia voy a tener y vale la pena ofertar?* (inteligencia de mercado).

24%

DE LAS ADJUDICADAS
TUVO UN SOLO
OFERENTE

14%

QUEDA DESIERTA
(NADIE GANA)

75%

ADJUDICADO VS
PRESUPUESTO
(MEDIANA)

43.000

LICITACIONES
ANALIZADAS (6 MESES)

Estos tres hallazgos sintetizan el valor: **existen nichos con demanda insatisfecha** (baja competencia y licitaciones desiertas que el organismo debe relicitarse), y empíricamente **se gana ofertando bajo el presupuesto publicado**.

2. Arquitectura del pipeline

FUENTE 1 · API Mercado Público (en vivo, con ticket, rate-limited)

main.py --solo-cierre	→	evolucion.csv	881.000 filas · serie 2020-2026
main.py --sin-detalle	→	licitaciones_anio.csv	56.000 filas · histórico liviano
main.py --estado activas	→	vigentes.csv	4.600 filas · enriquecidas 1 a 1

FUENTE 2 · Datos abiertos ChileCompra (masivos, mensuales, sin ticket)

intel.py	→	intel_licitaciones.csv	43.000 licitaciones resueltas
	→	intel_proveedores.csv	22.700 pares proveedor-rubro

TRANSFORMACIÓN · dbt + DuckDB (repo: github.com/DanielGoSi97/licitaciones-analytics-dbt)

staging/	→	limpieza, casteo de tipos, flags de competencia
marts/	→	esquema en estrella: fct_licitaciones, dim_proveedor, mart_competencia_por_rubro, mart_evolucion_mensual (+ tests de calidad)

CONSUMO

app.py (Streamlit) lee los 5 CSV cacheados y consulta la API en vivo únicamente para el detalle de la licitación que el usuario abre.

La decisión estructural más importante es la **estrategia de tres profundidades de extracción**. Consultar el detalle de una licitación cuesta 2-3 segundos por el rate-limit de la API; enriquecer las 881.000 filas históricas habría tomado un mes de cómputo. La solución: serie histórica con el mínimo de campos (rápida), enriquecimiento completo solo para las ~4.600 vigentes (3-4 horas), que es donde el detalle aporta valor. Y cuando se necesitó el "cómo terminaron" masivo —oferentes, adjudicaciones, proveedores ganadores— no se forzó la API: **se cambió de fuente** a los archivos mensuales de datos abiertos, que traen esa información pre-calculada.

3. Componentes del código

Archivo	Rol	Elementos clave
<code>client.py</code>	Conector con la API	Clase única <code>MercadoPublicoClient</code> ; reintentos con backoff exponencial (<i>tenacity</i>). Deliberadamente simple: solo trae JSON.
<code>models.py</code>	Capa de dominio	Modelos <i>pydantic</i> que validan la respuesta; mapeos de negocio (tipo LP/LE/L1, estados 5=Publicada, 7=Desierta, 8=Adjudicada); clasificadores heurísticos por palabras clave como <i>fallback</i> cuando no hay dato estructurado.
<code>main.py</code>	Extractor CLI (API)	CLI con rangos de fecha y 3 niveles de profundidad; manejo explícito del rate-limit (código 10500) con espera y reintento; tolerancia a fallas por fila (una licitación corrupta genera una fila básica, no aborta el proceso); <code>enrich_tender()</code> aplana el JSON anidado en ~25 campos tabulares.
<code>intel.py</code>	Pipeline de datos abiertos	Descarga zips mensuales (~270 MB de CSV, 110 columnas, nivel oferta-línea) y los colapsa en dos agregados livianos; filtros de calidad de datos : descarta líneas sobre \$20.000M y montos que excedan 300× el presupuesto (la fuente traía una "adjudicación" de \$1.700 billones, más que el PIB de Chile).
<code>app.py</code>	Dashboard Streamlit	Fusión de fuentes con deduplicación que prefiere la fila enriquecida; navegación tipo SPA con <code>st.session_state</code> y <code>@st.fragment</code> ; vista detalle consulta la API <i>en vivo</i> ; carga condicional (la app degrada con elegancia si falta una fuente); link directo a la ficha oficial de cada licitación.

4. La capa de inteligencia de mercado (diferenciación)

El buscador oficial de Mercado Público resuelve la búsqueda; las herramientas que agregan inteligencia (LicitaLab) son de pago, y el Observatorio ChileCompra mira probidad, no oportunidad comercial. El espacio libre era la **analítica para decidir dónde competir**, y eso es lo que agrega este proyecto:

- **Pestaña "Inteligencia de mercado"**: con filtros por sector oficial, rubro UNSPSC y región: % de adjudicaciones con oferente único, % de desiertas por sector, competencia promedio por rubro, brecha adjudicado/presupuesto y top 15 proveedores por monto (la competencia directa del usuario).
- **Panel "Competencia esperada" en cada detalle**: al abrir una licitación vigente, la app cruza su rubro UNSPSC con el histórico y muestra cuántas se resolvieron en ese rubro, promedio de oferentes, % con oferente único, % desierta y quiénes más ganan ahí — *antes* de decidir ofertar.

El cruce técnico: el rubro del detalle en vivo (primer nivel de la categoría UNSPSC de los ítems) se matchea contra `Rubro1` de los datos abiertos, normalizando a mayúsculas. Dos fuentes con esquemas distintos, unidas por la taxonomía común.

5. Decisiones de diseño defendibles

- **Mediana, no promedio, para el "monto típico"**: la distribución de montos tiene cola larga; un solo contrato hospitalario de \$16.000M arruina cualquier promedio.
- **Deduplicación con criterio**: una misma licitación aparece en varios meses de los datos abiertos (publicación vs adjudicación); se conserva el período más reciente, que trae el estado final. En el dashboard, ante duplicados se prefiere siempre la fila enriquecida.
- **Resiliencia transversal**: backoff exponencial, manejo del rate-limit 10500, degradación por fila, carga condicional de fuentes en la app.
- **Calidad de datos basada en evidencia**: los umbrales de limpieza (\$20.000M por línea, 300× el estimado) salieron de inspeccionar percentiles (p99,9 = \$3.400M) y validar los casos extremos uno a uno.
- **Validación automatizada**: la app completa se prueba headless con `streamlit.testing.v1.AppTest` (cero excepciones, pestañas y KPIs poblados) antes de cada cambio mayor.

6. Cómo presentar el proyecto

Elevator pitch (30 segundos)

"Construí un pipeline de datos y un dashboard sobre las compras públicas de Chile. Extrae licitaciones desde la API de Mercado Público y los datos abiertos de ChileCompra, los limpia y modela, y los presenta orientado a un proveedor que quiere venderle al Estado: qué está abierto, cuándo cierra y —lo diferenciador— cuánta competencia histórica hay en ese rubro. El 24% de las licitaciones se adjudica con un solo oferente y el 14% queda desierta: hay nichos con demanda insatisfecha que el buscador oficial no te muestra."

Narrativa para entrevista (problema → decisión → resultado)

1. **Problema:** el buscador oficial dice qué existe, pero no ayuda a decidir dónde competir; las herramientas que sí lo hacen son de pago.
2. **Restricción técnica:** rate-limit agresivo y detalle caro → estrategia de tres profundidades + segunda fuente masiva para el histórico.
3. **Modelamiento:** JSON anidado → tablas planas; nivel oferta-línea → nivel licitación; taxonomía UNSPSC real con fallback heurístico.
4. **Producto:** dashboard con persona definida y KPIs elegidos con criterio estadístico.
5. **Validación:** tests headless y sanity-checks de los agregados (ahí aparecieron los errores de digitación de la fuente, y se filtraron con umbrales justificados).

El insight que cierra la conversación

"La brecha mediana entre monto adjudicado y presupuesto es 75%: empíricamente, en este mercado se gana ofertando bajo el presupuesto publicado. Eso es un insight accionable que salió del pipeline, no de una opinión."

7. Capa analítica reproducible (dbt) y automatización CI/CD

Sobre los CSV crudos se construyó un **proyecto dbt** que separa la lógica de transformación del dashboard y la versiona en SQL: la limpieza deja de estar enterrada en pandas dentro de `app.py` y pasa a modelos declarativos, testeados y documentados. Corre localmente sobre **DuckDB** (cero credenciales, `dbt build` en segundos) y el mismo esquema se materializa en **BigQuery** cambiando solo el perfil — es *warehouse-agnostic*.

- **staging → marts (esquema en estrella):** 3 modelos de limpieza y 4 marts de negocio (`fct_licitaciones`, `dim_proveedor`, `mart_competencia_por_rubro`, `mart_evolucion_mensual`) sobre las ~43.000 licitaciones.
- **Tests de calidad declarativos:** 16 pruebas `not_null` / `unique` sobre claves de negocio y métricas críticas, que corren en cada build.
- **Actualización automática (GitHub Actions):** un workflow programado (semanal) y disparable a mano descarga datos frescos de ChileCompra con `intel.py`, ejecuta `dbt build` con sus tests y **commitea los CSV actualizados al repo** — todo en los servidores de GitHub, sin depender de ninguna máquina local.

El resultado es un **pipeline de datos con CI/CD**: ingesta → transformación versionada → tests → publicación, reproducible por cualquiera que clone el repositorio.

8. Operación

Acción	Comando	Frecuencia sugerida
Refrescar licitaciones vigentes (enriquecidas)	<code>python main.py --estado activas --salida vigentes.csv</code>	Diaria/semanal (3-4 h)

Refrescar histórico liviano	<code>python main.py --desde DDMMAAAA --hasta DDMMAAAA --sin-detalle --salida licitaciones_anio.csv</code>	Mensual
Refrescar inteligencia de mercado	<code>python intel.py --meses 6</code>	Mensual (~5 min, cuando ChileCompra publica el mes)
Reconstruir modelos analíticos + tests	<code>dbt build</code> (en <code>analytics_dbt/</code>)	Tras cada refresco de datos
Actualización automática end-to-end	GitHub Actions · workflow <code>refresh-data</code>	Automática (lunes) / manual
Levantar el dashboard	<code>streamlit run app.py</code> → <code>http://localhost:8501</code>	—

Proyecto personal de Daniel González Simpson · Fuentes: API de Mercado Público (api.mercadopublico.cl) y Datos Abiertos ChileCompra (datos-abiertos.chilecompra.cl) · Repo analítico: github.com/DanielGoSi97/licitaciones-analytics-dbt · Documento generado en junio de 2026.